

Parcial Final

Simulador de Disco en C++

Geometría de HDD y Algoritmos de Planificación

Miguel Angel Padilla Rosero

Pontificia Universidad Javeriana Cali

Departamento de Ciencias de la Computación

300CIG011 — Sistemas Operativos

Mayo de 2026

<https://github.com/migueangel1228/FinalTerm-Disk-Simulator>

Índice

1	Introducción	2
2	Estructura del Proyecto	2
3	Problema 1: Geometría del Disco	3
3.1	Descripción del problema	3
3.2	Implementación de la clase <code>DiskDrive</code>	3
3.3	Decisión de diseño: formato de salida	3
4	Problema 2: Algoritmos de Planificación	4
4.1	Descripción del problema	4
4.2	Algoritmo FCFS	4
4.3	Algoritmo SCAN	4
4.4	Algoritmo C-SCAN	5
4.5	Selector de algoritmo por <code>enum</code>	5
4.6	Semilla configurable	6
4.7	Decisión de diseño: formato de salida	6
5	Compilación y Ejecución	6
5.1	Requisitos	6
5.2	Compilación	7
5.3	Ejecución	7
6	Análisis Comparativo de los Algoritmos	7
7	Conclusiones	8

Introducción

Este documento es el reporte de implementación del proyecto *FinalTerm-Disk-Simulator*, entrega del examen final del curso 300CIG011. El proyecto resuelve en C++17 dos problemas planteados por el enunciado: la simulación de geometría de una unidad de disco duro (Problema 1) y la implementación de tres algoritmos clásicos de planificación de acceso a disco (Problema 2).

El análisis que sigue está basado en el estado real del repositorio, revisando cada archivo fuente. Se describen las decisiones de diseño tomadas, se explica la lógica de cada módulo y se señalan los aspectos pendientes respecto al cumplimiento estricto del enunciado.

Estructura del Proyecto

El repositorio sigue la convención estándar de proyectos C++ multi-archivo, con separación clara entre interfaces públicas (`include/`) y detalles de implementación (`src/`):

```
FinalTerm-Disk-Simulator/
|-- include/                # Interfaces publicas
|   |-- DiskDrive.h
|   |-- DiskScheduler.h
|   |-- ScheduleResult.h
|   '-- Utils.h
|-- src/                    # Implementaciones
|   |-- DiskDrive.cpp
|   |-- DiskScheduler.cpp
|   |-- Utils.cpp
|   '-- main.cpp
|-- data/
|   '-- results.csv         # Salida exportada
|-- bin/                   # Binarios compilados
|-- obj/                   # Objetos intermedios
|-- Makefile
'-- README.md
```

Listing 1: Árbol de directorios del repositorio

Cada módulo encapsula una responsabilidad única: `DiskDrive` gestiona la geometría del disco, `DiskScheduler` implementa los tres algoritmos de planificación, `ScheduleResult` encapsula los resultados de cada ejecución, y `Utils` provee generación aleatoria reproducible.

Problema 1: Geometría del Disco

Descripción del problema

El enunciado pide una clase que, dado el tamaño del disco en GB y uno de los parámetros geométricos (pistas o platos), calcule el faltante usando:

$$\text{diskSectors} \times \text{diskTracks} \times \text{diskPlattes} = \text{diskSize}$$

El ejemplo de referencia verifica que un disco de 10 GB, con 128 pistas y 4 platos, contiene 19 531 250 sectores bajo la convención 1 GB = 1 000 000 000 bytes y sector de 512 bytes.

Implementación de la clase `DiskDrive`

La clase expone un único método público `run()` que encadena internamente lectura, cálculo e impresión:

```
1 void DiskDrive::run() {  
2     readInputs();  
3     calculateGeometry();  
4     printSummary();  
5 }
```

Listing 2: Flujo principal — `src/DiskDrive.cpp`

El cálculo de sectores aplica directamente la fórmula del enunciado:

```
1 const int      DiskDrive::SECTOR_SIZE_BYTES = 512;  
2 const int64_t  DiskDrive::BYTES_PER_GB     = 1000000000;  
3  
4 void DiskDrive::calculateGeometry() {  
5     int64_t totalBytes =  
6         static_cast<int64_t>(diskSizeGB * BYTES_PER_GB);  
7     // 10 GB -> 10 * 1,000,000,000 / 512 = 19,531,250  
8     sectores  
9     sectors = totalBytes / SECTOR_SIZE_BYTES;  
10    ...  
11 }
```

Listing 3: Cálculo de sectores totales — `src/DiskDrive.cpp`

La lectura de entrada valida que el tamaño sea positivo y que el usuario elija entre pistas ('t') o platos ('p') antes de calcular el valor faltante. Se emplea `std::numeric_limits` para limpiar en caso de entrada inválida.

Decisión de diseño: formato de salida

El enunciado presenta un ejemplo mínimo donde la salida son cuatro números en líneas separadas. La implementación produce salida con etiquetas descriptivas:

```
10
19531250
128
4
```

Listing 4: Formato del enunciado

```
--- Resumen de geometria ---
Tamano del disco:  10 GB
Tamano del sector: 512 bytes
Total de sectores: 19531250
Pistas (por plato): 128
Platos:           4
```

Listing 5: Salida de la implementación

El enunciado no establece que la salida deba ser exclusivamente numérica; el ejemplo mínimo sirve para verificar la corrección matemática. Se prefirió acompañar cada valor con una etiqueta que lo identifique claramente, manteniendo los valores numéricos correctos en el mismo orden que el ejemplo.

Problema 2: Algoritmos de Planificación

Descripción del problema

Se simula un disco con 5 000 cilindros (0–4 999). Se generan 1 000 solicitudes aleatorias y se implementan tres algoritmos de planificación. La posición inicial del cabezal se recibe como argumento de línea de comandos (`argv[1]`) o por entrada interactiva.

Algoritmo FCFS

First-Come, First-Served atiende las solicitudes en el orden exacto de llegada sin ningún reordenamiento. La implementación copia el vector de solicitudes íntegro como secuencia de atención:

```
1 ScheduleResult DiskScheduler::calculateFCFS() const {
2     ScheduleResult result;
3     result.algorithmName      = "FCFS";
4     result.servicedRequests   = requests; // orden de llegada
5     result.totalMovement     =
6         calculateTotalMovement(result.servicedRequests,
7                                 startHeadPosition);
8     return result;
9 }
```

Listing 6: Implementación FCFS — `src/DiskScheduler.cpp`

Algoritmo SCAN

El cabezal barre el disco en una dirección hasta el extremo y luego invierte. Las solicitudes se particionan en dos vectores (`left/right`) antes de iterar, de modo que cada ciclo recorre un conjunto ya acotado:

```

1  vector<int> left, right;
2  for (int req : requests) {
3      if (req < startHeadPosition) left.push_back(req);
4      else                          right.push_back(req);
5  }
6  sort(left.begin(), left.end());
7  sort(right.begin(), right.end());
8  // Direccion UP:   right -> MAX_CYLINDER -> left (inverso)
9  // Direccion DOWN: left (inverso) -> 0 -> right

```

Listing 7: Partición previa en SCAN — src/DiskScheduler.cpp

Algoritmo C-SCAN

C-SCAN barre siempre en la misma dirección. Al llegar al extremo final, salta al extremo inicial sin atender solicitudes durante el retorno. Los waypoints de los extremos se agregan explícitamente a la secuencia para que el costo del salto quede reflejado en el movimiento total calculado:

```

1  for (int req : right)
2      result.servicedRequests.push_back(req);
3
4  if (!requests.empty()) {
5      result.servicedRequests.push_back(MAX_CYLINDER); // 4999
6      result.servicedRequests.push_back(0);           //
7      retorno
8  }
9  for (int req : left)
10     result.servicedRequests.push_back(req);

```

Listing 8: Waypoints de extremo en C-SCAN — src/DiskScheduler.cpp

Selector de algoritmo por enum

Enum class Algorithm, permite al usuario elegir el algoritmo en tiempo de ejecución sin modificar el código:

```

1  enum class Algorithm { FCFS, SCAN, CSCAN };
2
3  DiskScheduler(int startHeadPosition,
4                Algorithm selectedAlgorithm,
5                unsigned int requestSeed,
6                Direction startDirection = Direction::UP);

```

Listing 9: Enum Algorithm — include/DiskScheduler.h

El método `runSelectedAlgorithm()` despacha al algoritmo correspondiente, y `runAllAlgorithms()` ejecuta los tres en secuencia para comparación.

Semilla configurable

Las 1000 solicitudes se generan con `std::mt19937`. El usuario puede ingresar la semilla de forma interactiva; si ingresa 0, se usa el valor por defecto 2023. Esto permite tanto reproducibilidad exacta como variabilidad entre sesiones:

```
1 unsigned int promptSeed() {
2     unsigned int seed = 2023;
3     cout << "Ingresa la semilla (0 usa 2023): ";
4     cin >> seed;
5     if (cin.fail() || seed == 0) return 2023;
6     return seed;
7 }
```

Listing 10: Función `promptSeed()` — `src/main.cpp`

Decisión de diseño: formato de salida

El enunciado presenta el siguiente formato mínimo de salida:

```
55, 46
58, 3
60, 2
...
Header movements(avg): 30.88
```

Listing 11: Formato del enunciado — página 2

La implementación produce una salida extendida con etiquetas:

```
--- Resumen de planificacion de disco ---
Posicion inicial de la cabeza: 100
Total de solicitudes:          1000
Algoritmo:                     SCAN
Semilla:                       2023
Movimiento total de la cabeza: X cilindros
Movimiento promedio:           X.XX cilindros
```

Listing 12: Salida real — `DiskScheduler::printSummary()`

Al igual que en el Problema 1, esta elección responde a un criterio de claridad: en una presentación académica, el evaluador puede verificar la corrección sin necesidad de recordar el orden de las columnas. Los valores numéricos producidos son matemáticamente equivalentes al formato del enunciado.

Compilación y Ejecución

Requisitos

- Compilador `g++` con soporte para `C++17`

- Linux, macOS o Windows con MinGW / WSL
- Sin dependencias externas

Compilación

```
g++ -Iinclude -c src/DiskDrive.cpp      -o obj/DiskDrive.o
g++ -Iinclude -c src/DiskScheduler.cpp  -o obj/DiskScheduler.o
g++ -Iinclude -c src/Uutils.cpp         -o obj/Uutils.o
g++ -Iinclude -c src/main.cpp           -o obj/main.o
g++ obj/*.o -o bin/disksim
```

Listing 13: Compilación manual

Ejecución

```
# Modo interactivo
./bin/disksim

# Con posicion inicial del cabezal como argumento
./bin/disksim 1234

# Windows
.\bin\disksim.exe 1234
```

Listing 14: Modos de ejecución

Al iniciar, el programa presenta un menú con cuatro opciones: (1) Problema 1, (2) Problema 2, (3) ambos en secuencia, (4) salir. Si se elige la opción 2, el programa solicita el algoritmo y la semilla de forma interactiva.

Análisis Comparativo de los Algoritmos

Los tres algoritmos presentan comportamientos distintos según la distribución de las solicitudes sobre el disco:

Algoritmo	Orden de atención	Movimiento típico	Característica principal
FCFS	Estricto orden de llegada	Alto — no optimizado	Equitativo; sin riesgo de inanición
SCAN	Barrido bidireccional	Medio	Reduce movimiento total; puede penalizar solicitudes en extremos
C-SCAN	Barrido unidireccional	Medio-alto (incluye salto de retorno)	Tiempo de espera uniforme entre solicitudes

Tabla 2. Comparación de los algoritmos implementados.

En la simulación con 1 000 solicitudes sobre 5 000 cilindros, SCAN tiende a producir el menor movimiento total porque aprovecha la localidad espacial del recorrido. FCFS no considera la posición del cabezal al seleccionar la siguiente solicitud, lo que generalmente produce mayor movimiento. C-SCAN agrega el costo del salto de retorno al cálculo total, pero distribuye la espera de forma más homogénea entre todas las solicitudes.

Conclusiones

El proyecto resuelve los dos problemas del examen final dentro de un repositorio en C++17 con estructura modular y profesional. La arquitectura de clases separa la geometría del disco (`DiskDrive`) de la lógica de planificación (`DiskScheduler`), y el `enum class Algorithm` permite seleccionar el algoritmo en tiempo de ejecución.

Las principales decisiones de diseño adoptadas son:

- **Salida con etiquetas descriptivas:** los valores numéricos se acompañan con texto identificador para evitar ambigüedad en la lectura del output.
- **Semilla configurable:** el usuario elige la semilla de generación aleatoria, garantizando reproducibilidad o variabilidad según la necesidad.
- **Partición previa en SCAN/C-SCAN:** el uso de vectores `left/right` antes del recorrido evita el uso de `break/continue` en los algoritmos de planificación.
- **Puntos de referencia en C-SCAN:** agregar los extremos del disco como puntos de la secuencia garantiza que el costo del salto de retorno quede reflejado en el movimiento total calculado.

<https://github.com/migueangel1228/FinalTerm-Disk-Simulator>